DG INTERNS HUB

CYBERSECURITY INTERNSHIP

WEEK 8 ASSESSMENT REPORT

ADVANCED VAPT – ATTACK CHAIN & FULL EXPLOITATION PROJECT

Submitted By:
Amaan Fayaz

Internship Domain:
Cybersecurity

Target Application:
Damn Vulnerable Web Application (DVWA)

Tools Used:

- Burp Suite Community Edition
- Docker Desktop
- Web Browser

Security Level:
Medium

Submission Date: 28 October 2026

Introduction: Vulnerability Assessment and Penetration Testing (VAPT) is a cybersecurity process used to identify, analyze, and validate security weaknesses in applications and systems.

This project focuses on Advanced VAPT, with particular emphasis on understanding how multiple security weaknesses may combine to create a serious security risk.

The project uses the Damn Vulnerable Web Application (DVWA), a deliberately vulnerable web application designed for security testing and learning. Burp Suite Community Edition is used to inspect HTTP requests and responses and understand how the application communicates with the browser.

2. Objectives of the Project:

The main objectives of this assessment are:

1. To understand the process of advanced vulnerability assessment and penetration testing.
2. To identify and map the entry points of a web application.
3. To understand HTTP requests and responses using Burp Suite.
4. To investigate parameter manipulation and access control weaknesses.
5. To examine session security and business logic validation.

3. Target Application and Scope

Target Application: Damn Vulnerable Web Application (DVWA)

Local Application Address: http://localhost:4280

Testing Environment: Local Docker container running on macOS

Testing Tool: Burp Suite Community Edition

Configured Security Level: Medium

4. Tools and Technologies Used

4.1 DVWA: Damn Vulnerable Web Application is a deliberately vulnerable web application used to learn web application security testing. It provides a controlled environment for understanding common vulnerabilities.

4.2 Burp Suite Community Edition: Burp Suite is a web security testing tool used to intercept, inspect, and analyze HTTP requests and responses. Its Proxy and Repeater features help security testers understand application behavior and validate potential vulnerabilities.

4.3 Docker Desktop: Docker Desktop is used to run the DVWA application in a container on the MacBook. This provides an isolated environment for practical assessment.

4.4 Web Browser: The browser is used to access DVWA and interact with its pages and forms. Burp Suite's built-in browser is used to capture application traffic.

4.5 macOS: macOS is the host operating system used to run Docker Desktop and Burp Suite for this assessment.

5. Lab Environment Setup: The first phase of the assessment involved preparing the tools and configuring the local testing environment.

Step 1: Checking Docker Installation

Docker was already installed on the MacBook. The Docker version command was executed in Terminal to verify its installation.

Step 2: Checking Existing Containers

The Docker container list was inspected to locate the existing DVWA container.

The container was identified with the name dvwa and the image vulnerables/web-dvwa.

Step 3: Starting DVWA

The existing DVWA container was started using Docker. Its running status was verified using the Docker container listing command.

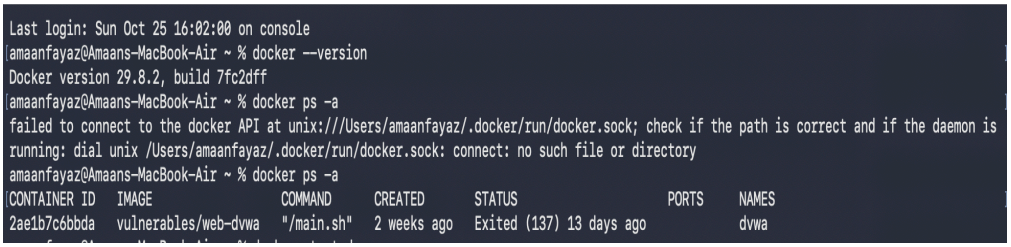


Figure 1: Verification of Docker installation and identification of the existing DVWA container.

This screenshot shows the Docker version installed on macOS and the identification of the existing DVWA container. The container was initially found in a stopped state before it was started.

Step 4: Accessing the Application

The DVWA application was opened in the browser using its local address:

http://localhost:4280

The application dashboard was accessible after logging in.

Step 5: Configuring Security

The DVWA security setting was changed to Medium for the assessment.

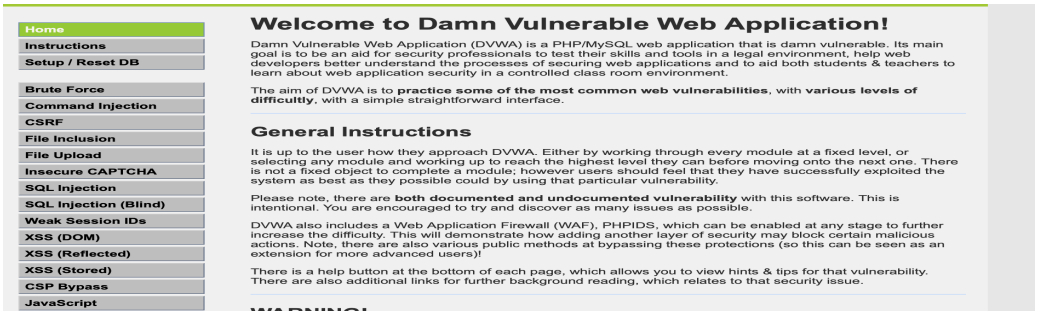


Figure 2: Successful access to the locally hosted DVWA application, showing the homepage and available testing modules.

This screenshot shows the DVWA homepage after successful access to the local web application. It displays the available modules used for learning and testing web application security.

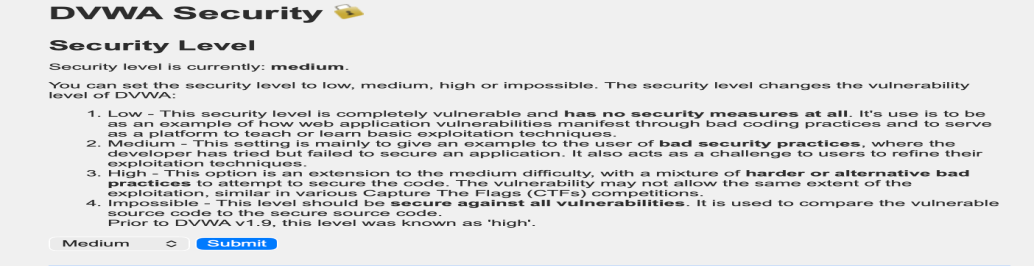


Figure 3: Verification of DVWA security level set to Medium for controlled vulnerability testing.

This screenshot confirms that the DVWA security level is set to Medium. This configuration provides the environment for the planned vulnerability testing activities.

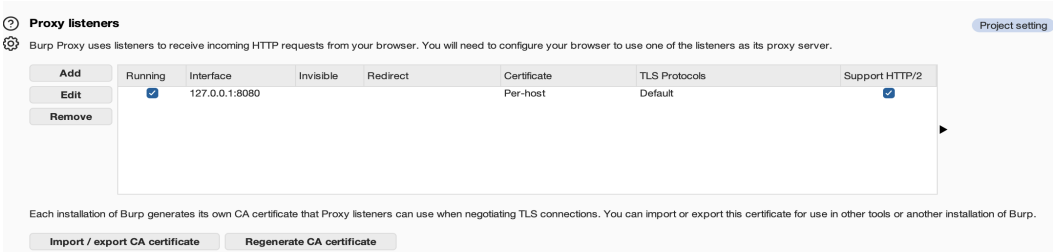


Figure 4: Verification of the active Burp Suite Proxy listener at 127.0.0.1:8080.

This screenshot shows that the Burp Suite Proxy listener is active on 127.0.0.1:8080. The listener is configured to receive HTTP requests from the browser for inspection during the assessment.

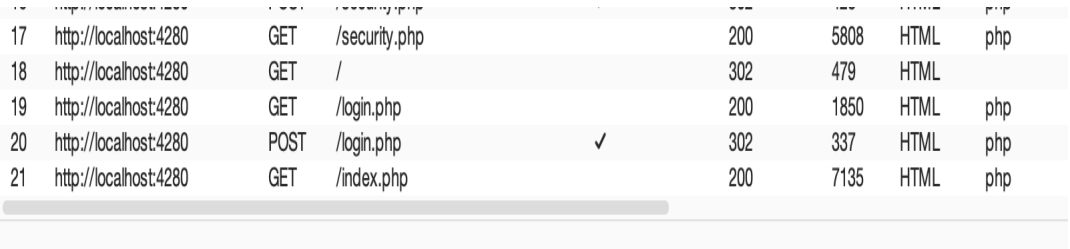


Figure 5: HTTP request capture in Burp Suite showing DVWA login activity and subsequent page navigation.

Burp Suite captured HTTP requests generated while accessing DVWA. The history shows GET and POST requests, including the login request, helping us understand how the browser communicates with the web application.

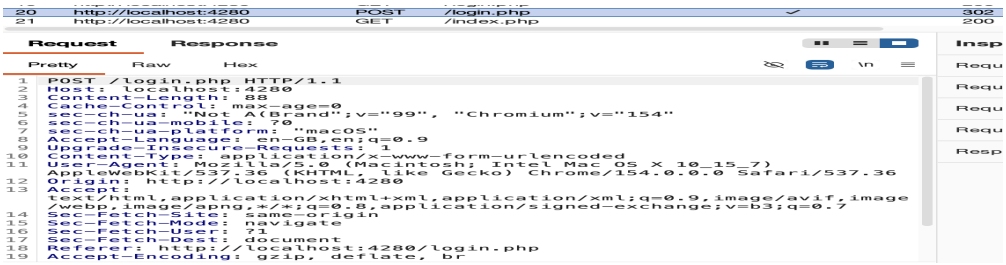


Figure 6: Inspection of the DVWA login request using Burp Suite.

The POST request to /login.php was inspected using Burp Suite. The request headers show details such as the host, content type, and browser information. This analysis helps explain how login form data is transmitted to the web application for authentication.

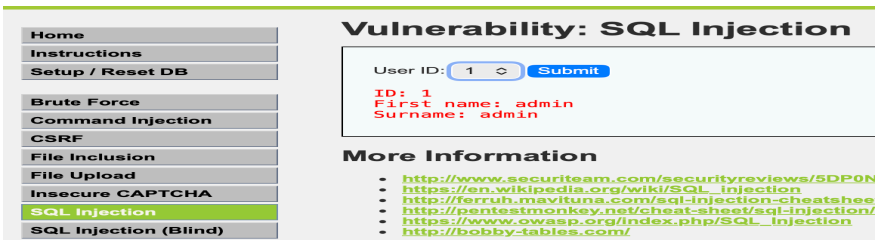


Figure 7: Initial SQL Injection Testing in DVWA

The SQL Injection module was tested by entering User ID 1. The application returned the corresponding user information, showing how the application processes user input and retrieves records from its database. This result provides a baseline for further security testing.



Figure 8: Successful SQL Injection Testing in DVWA at Low Security

The SQL Injection module was tested using the input ' OR '1'='1. The application returned multiple user records, including admin and other users, instead of restricting the result to a single record. This demonstrates that the application is vulnerable to SQL injection at the Low security level. Parameterized queries and proper input handling can help prevent this vulnerability.

Objective: explaining that the assessment aims to combine multiple weaknesses to understand their combined security impact.

Target Application: selected DVWA running locally on your Mac as your testing environment.

1. Attack Surface Mapping

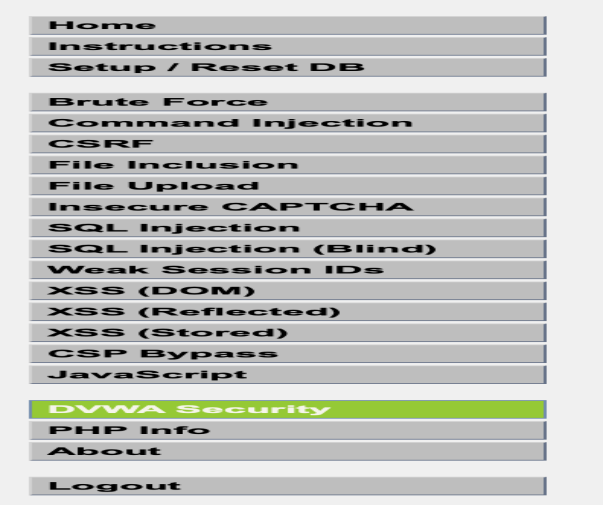


Figure 9: Identification of the DVWA Attack Surface

The DVWA application was examined to identify its main entry points for security testing. These include SQL Injection, Cross-Site Scripting (XSS), Brute Force, Command Injection, File Upload, and other modules. These entry points help map the application's attack surface and identify areas for further vulnerability assessment.

Application Flow Mapping

The DVWA application was explored by logging in and visiting its security testing modules.

Application Flow: Login → Home Page → Testing Modules → Input Submission → Server Response.

Business Logic Testing

Different user ID values were tested in DVWA. IDs 1 and 2 displayed user records, while ID 999 did not display a user record. This test examined input handling but did not confirm an IDOR or business logic vulnerability.

Burp Repeater Test: At Medium security, changing the ID parameter from 1 to 2 returned Gordon Brown's record. This confirmed that the parameter affected the response, but did not independently establish an authorisation vulnerability.

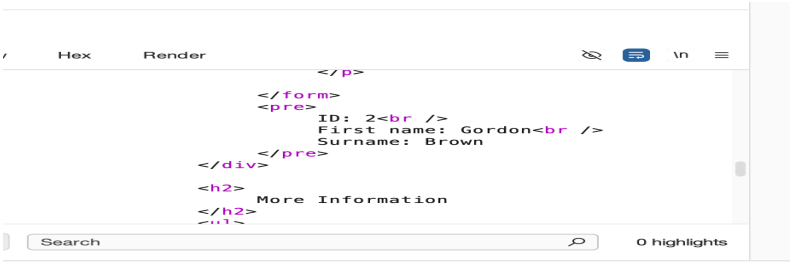


Figure 10: Burp Suite Repeater Testing at Medium Security (ID 2 – Gordon Brown)

IDOR Testing: No separate profile or account feature was available in the tested DVWA modules. Therefore, an IDOR vulnerability could not be confirmed and requires further testing with a suitable application.



Figure 11: Successful Reflected Cross-Site Scripting (XSS) Testing in DVWA at Low Security

The Reflected XSS vulnerability was tested in DVWA using a JavaScript payload. The browser displayed an alert message saying "XSS Test," confirming that the supplied input was executed as JavaScript. This demonstrates that the application is vulnerable to reflected XSS at the Low security level. Context-appropriate output encoding and proper input handling can help prevent this vulnerability.



Figure 12: Successful Stored Cross-Site Scripting (XSS) Testing in DVWA

The Stored XSS module was tested in DVWA using a JavaScript payload. The browser displayed an alert message saying "Stored XSS Test," indicating that the submitted content was stored and later executed when displayed.

Parameter Tampering (Changing Request Values in Burp Suite

The id parameter was changed from 1 to 2 using Burp Suite Repeater. The modified request returned HTTP 200 OK. This confirms that the application processed the request, but further examination of the response content is necessary to determine which record was returned and whether any access control weakness exists.



Figure 13: Parameter Tampering Test Using Burp Suite Repeater using id 2

The id parameter was changed from 1 to 2. Burp Suite Repeater was used to send the modified request and inspect the server response. The server returned HTTP 200 OK.



Figure 14: Testing a Nonexistent ID Parameter using id 999

The id parameter was changed to 999 to test how the application handled a potentially nonexistent user ID. The server returned HTTP 200 OK. Further response-content analysis is required to confirm whether a matching record was returned.

2. Advanced Testing (Logic-Based)

Business Logic Flaws (Bypassing Steps and Skipping Validation)

Business logic testing checks whether an application follows the correct sequence of steps and validates user actions properly.

Session Management Testing



Figure 15: Session Handling Test Using Burp Suite Repeater

A request was sent to DVWA after removing the session cookie. The server returned HTTP 302 Found and redirected the request to the login page. This indicates that authentication was required to access the tested resource.

SQL Injection Testing at Medium Security

Burp Suite Repeater was used to test the DVWA SQL Injection page. With User ID 1, the response displayed the admin record. With User ID 2, it displayed Gordon Brown's record. Both requests returned HTTP 200 OK. These results show that the application returned the requested records, but they do not independently confirm SQL injection, IDOR, or an authentication bypass.



Figure 16: Burp Suite Repeater Response for User ID 2 at Medium Security

Invalid User ID Test: The User ID was changed to 999 in Burp Suite Repeater. The server returned HTTP 200 OK, but no matching user record was displayed. This indicates that the application did not display a record for the supplied ID. The HTTP status alone does not indicate a vulnerability.

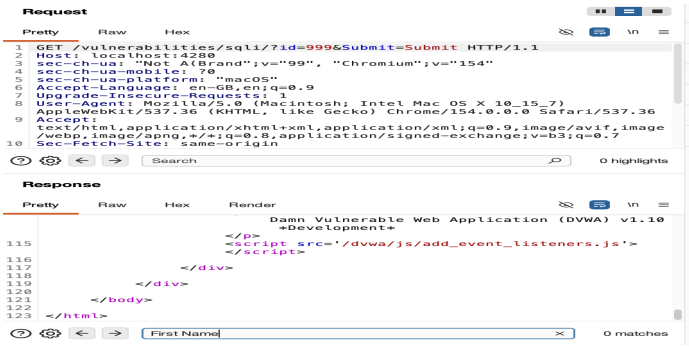


Figure 17: Burp Suite Response to Invalid User ID 999 at Medium Security

Reflected XSS Testing at Medium Security

The Reflected XSS module was tested at Medium security using ordinary text as input. The application displayed “Hello Week8Test” on the page. This confirmed that ordinary input was reflected, but it did not demonstrate successful XSS or script execution.



Figure 18: Reflected XSS Input Handling at Medium Security

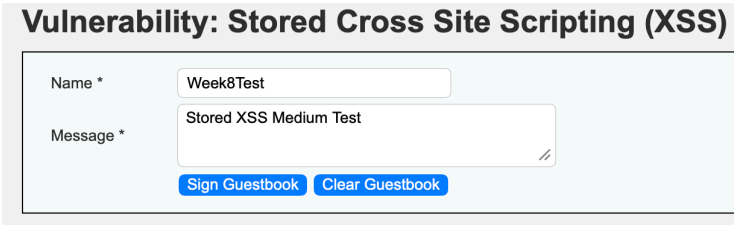


Figure 19: Stored XSS Test Input at Medium Security

Stored XSS Testing at Medium Security

The Stored XSS module was tested in the local DVWA environment with the security level set to Medium. A test message was entered into the guestbook and submitted for testing. This screenshot shows the input form and the test message used. The JavaScript alert was observed during testing, but the alert itself is not shown in this screenshot.

Test IDOR (Insecure Direct Object Reference)

Result: Preliminary testing was performed by changing user IDs in the DVWA SQL Injection module. Different records appeared for IDs 1 and 2, while ID 999 returned no record. Because this test uses the SQL Injection module, it does not independently establish an IDOR vulnerability. Further testing of object-level authorization is required.

3. Attack Chain Analysis

The purpose of attack chain analysis is to understand how multiple vulnerabilities can combine to increase security risks. In this assessment, SQL Injection and Reflected XSS were tested at Low security, while Stored XSS was also tested at Medium security. Parameter tampering and session handling were examined using Burp Suite. However, a complete attack chain leading to account takeover or privilege escalation was not confirmed.

Attack Chain Diagram and Explanation

DVWA Application → Attack Surface Mapping → SQL Injection Testing → XSS Testing → Parameter Tampering → Session Management Testing → Findings and Remediation.

Attack Chain Steps

Combined Security Impact

SQL injection and reflected XSS were demonstrated in DVWA at Low security. Stored XSS was also tested. These findings highlight input-handling risks, but a complete attack chain involving account takeover or privilege escalation was not confirmed.

Step 1: Identify Entry Points
DVWA pages and input fields were explored using Burp Suite.

Step 2: SQL Injection Testing
SQL Injection testing at Low security displayed multiple user records.

Step 3: XSS Testing
Reflected and Stored XSS tests triggered JavaScript alerts at Low security.

Step 4: Parameter and Session Testing
User ID parameters were modified, and a request without a session cookie was redirected to the login page.

Step 5: Combined Impact
The tests demonstrated individual weaknesses, but a complete attack chain leading to account takeover or privilege escalation was not confirmed.

4. Findings, Risk Levels, and Remediation

SQL Injection

Risk: High (preliminary)

Finding: Testing at Low security displayed multiple user records.

Remediation: Use parameterised queries and validate inputs.

Reflected XSS

Risk: Medium (preliminary)

Finding: The test at Low security displayed submitted input.

Remediation: Encode output and validate inputs.

Stored XSS

Risk: High (preliminary)

Finding: A JavaScript alert appeared during testing at Medium security.

Remediation: Sanitise stored content and encode output.

Parameter Tampering

Result: Needs Further Testing

Finding: IDs 1 and 2 displayed different records; ID 999 displayed no matching record. This does not confirm an authorisation flaw.

Remediation: Check authorisation on the server.

Session Management

Result: Authentication Required

Finding: A request without a session cookie redirected to the login page.

Remediation: Use secure session handling and cookie settings.

5. Risk Impact Analysis

- SQL Injection: May expose user records and sensitive information.
- Reflected XSS: May execute malicious JavaScript in a user's browser.
- Stored XSS: May affect users who view the affected content.
- Business Impact: These weaknesses may cause data exposure and reduce user trust.

These impacts describe potential risks. Testing confirmed SQL Injection at Low security and Reflected XSS at Low security. A complete attack chain, account takeover, and privilege escalation were not confirmed.

6. Remediation Strategy

- SQL Injection: Use parameterized queries and validate inputs.
- XSS: Sanitize user input and apply proper output encoding.
- Access Control: Verify user permissions on the server before allowing access to data.
- Session Security: Use secure cookies and invalidate sessions after logging out.
- Overall Security: Regularly test the application and fix identified vulnerabilities.

7. Conclusion

This assessment helped me understand basic web application security testing using DVWA and Burp Suite. I tested SQL Injection, Reflected XSS, Stored XSS, parameter tampering, and session handling. I learned how to identify potential vulnerabilities, analyze responses, and suggest security improvements. However, a complete attack chain leading to account takeover or privilege escalation was not confirmed.

8. Limitations

- Testing was performed in a local DVWA lab.
- Some tests were performed at Low security, while Stored XSS was also tested at medium security.
- Parameter tampering did not confirm an access-control vulnerability.
- A complete attack chain leading to account takeover or privilege escalation was not demonstrated.

9. References

1. DVWA (Damn Vulnerable Web Application) — Local testing environment.
2. PortSwigger — Burp Suite documentation.
3. OWASP — Cross-Site Scripting (XSS) Prevention Cheat Sheet.
4. OWASP — SQL Injection Prevention Cheat Sheet.
5. OWASP — Web Security Testing Guide.